

So what are the types of storage systems and where can you use them? Let's dig just a little bit deeper.

NFS (Network File System): The simplest of ways to organise data is to store it in flat files and allow others on the network to access them using network shares. This works well enough for home users or small teams working together. It is simple and elegant but when users' needs increase and the question of 'who can access what' raises its head, you need some kind of access control.

Domains and ACL (Access Control List): When a plain NFS starts causing problems, you need something that allows you to strictly control the access to data and ensure that it is secure enough. Though not an open source solution, Windows Server with its ADS (Active Directory Services) is the closest to solving such problems. But that service can only work in a controlled and closed environment like an enterprise set-up. If you want to go ahead and manage even more data and users, you will need something that is distributed in nature and can be controlled much more easily. Traditional SQL databases may come to your rescue.

RDBMS (Relational Database Management System): There are not too many ready-made easy-to-use solutions that allow you to store huge amounts of data on data stores once we grow beyond what our home networks can offer. We are talking about access controls, scalability, accessibility, failovers, etc. RDBMS addresses the constraints of this level. You can have data storage distributed geographically and with a simple layer of programming above it, yet make it look like all the data is at one place. None of your users need to know how you have distributed your data. However, there are problems with this approach that need to be resolved. These are mostly related to the size of a single dataset. For example, MySQL's BLOB data type can store a maximum of 4 GB of data in one row and PostgreSQL's byte data type can store only 1 GB. You cross that limit and you have to find out ways to store larger files by yourself. Taking care of distribution on a RDBMS is not very simple.

Also, since we are talking about 'data stores' and not just file stores, an RDBMS is very good for storing relational data that can be arranged into rows and columns. The part where you would have to be vigilant is when mixing BLOBs and relational data. Mixing one into the other is normally not considered a very good design decision.

Document and key-value stores: Document stores allow non-relational data with dynamic fields to be stored elegantly. Key-value stores are used to store a piece of data (in whatever format you want since that choice does not affect the storage system) and identify it with a key. Search capabilities are very limited in key-value stores and different document stores put a lot of constraints on them in terms of data sizes, scalability, speeds and concurrency of access. These solutions focus on distributed access from the word go, which is very helpful in the long run. This is one area in which they fare better than an RDBMS. In the limited space that we have here, it is not possible to talk in detail about all the options

but, broadly speaking, here are the pros and cons of different types of solutions in key-value stores and document stores.

Key-value (KV) stores: These are mostly used when you have to search for the data elsewhere, and the only thing you query the KV store with is the key of the data you want. In most cases, KV stores do not handle replication, backups and scalability.

Document stores: These allow you to store data that can be grouped into tables but has too much of irregularity for an RDBMS to be used as the store. They can also offer a virtual file system, which can be used to automatically distribute the data across the globe (it would be your responsibility to properly set it up) and make it appear like a normal file-system. MongoDB, a famous document store, features one such file system called GridFS. Automated replications, backups, network partitioning, zonal-data awareness (the server closest to the requesting client serves the data) and automated data distribution on predefined keys are some of the many perks offered.

Pure distributed data stores: One of the most famous NoSQL solutions, Hadoop, falls in this category. It is just a pure data store that allows you to store files of any size. HBase, a NoSQL solution, is based on Hadoop and is a structured column-oriented database with the capability to run sophisticated analytical queries.

Hybrid data stores: Think of the data storage solution being built on two databases instead of just one. What if you store large files in MongoDB's GridFS and store its ObjectId in a MySQL database? You can store these ObjectIds in any way that suits your use-case. In this case, you would have the distributed capabilities of MongoDB along with the rich and well known querying mechanism of MySQL, both at the same time. You search for what you want in MySQL and get the key. Then you can use the key to get data from MongoDB—it's simple and very powerful. Of course, MongoDB is just an example. Hadoop, a KV store or a customised application built for a particular purpose would also work very well.

I do not say that any one of the above methods is better than the other. I've merely listed them in the order of complexity, beginning with the least complex. If you want to store your videos and music for your family to view and hear, an NFS is a much better solution than installing Hadoop or MongoDB on one of your computers. If you have to manage a lot of data with more granularity while trying to curb redundancy and enforce strict access controls, it would be best to choose an RDBMS solution. If all you care about can be summed up in two words—reliability and scalability—then a hybrid solution would be a great thing to go with.

There is no perfect solution and as you progress from simplicity to sophistication, your challenges will change. The same is true with data stores. 

By: Vaibhav Kaushal

The author is a Web developer staying in Bengaluru who loves writing for technology magazines. He can be reached at vaihbavkaushal123@gmail.com.